O.O.P Approach in C#.Net



Soyeb Ghachi

Technical Trainer | Mentor |SME

Overview



Introduction to Object Oriented Programming

Features of Object-Oriented Programming

-What is Abstraction and Encapsulation?

-What is Inheritance and Types of Inheritance?

-What is Polymorphism?

What is Interface?

What is Constructor?

What is properties?

Access modifiers

Introduction to O.O.P

The object oriented programming (OOP) is a programming model where Programs are organized around object and data rather than action and logic .OOP allow decomposition of a problem into a number of entities called Object and then builds data and objects around these objects.

- **Class:**

A class is the core of any modern Object Oriented Programming language such as C#. In OOP languages it is must to create a class for representing data.

Class is a blueprint of an object that contains variables for storing data and functions to performing operations on these data.

Class will not occupy any memory space and hence it is only logical representation of data.

```
using System;

public class Person
{
    // Fields
    private string name;
    private int age;

    // Constructor
    public Person(string name, int age)
    {
        this.name = name;
        this.age = age;
    }

    // Methods
    public void SayHello()
    {
        Console.WriteLine("Hello, my name is " +
name + " and I am " + age + " years old.");
    }
}
```

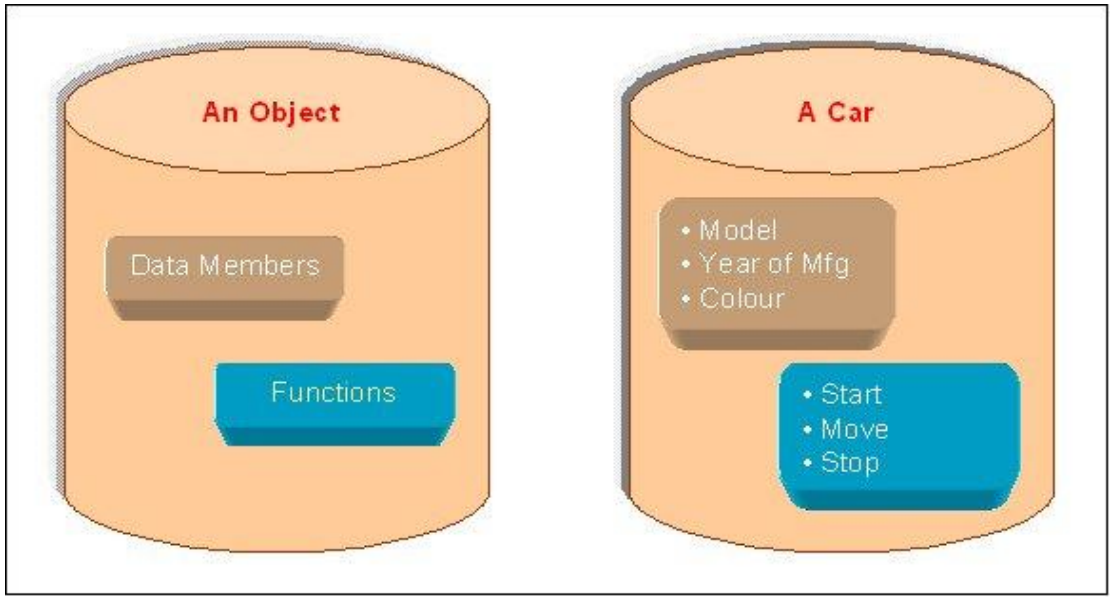
```
class Program
{
    static void Main(string[] args)
    {
        // Create an instance of the Person class
        Person person = new Person("John", 30);

        // Call the SayHello method
        person.SayHello();

        // Output: Hello, my name is John and I am 30
years old.
    }
}
```

- **Object:**

Objects are the basic run-time entities in an object oriented system. They may represent a person, place or any item that the program has to handle. "Object is a Software bundle of related variable and methods. “Object is an instance of a class”.



About Class and Object:

- Class will not occupy any memory space.
- Hence to work with the data represented by the class you must create a variable for the class, which is called as an object.
- When an object is created by using the keyword new, then memory will be allocated for the class in heap memory area. which is called as an instance.
- When an object is created without the keyword new, then memory will not be allocated in heap.

```
class Employee
{
}
```

Syntax to create an object of class Employee:-

```
Employee objEmp = new Employee();
```

All the programming languages supporting object-oriented Programming will be supporting these four main concepts:

- **Abstraction: It is mechanism to show only relevant data to the user**. Abstraction helps in managing complexity by providing a higher level of abstraction, allowing developers to work with simplified models that hide the underlying implementation details.

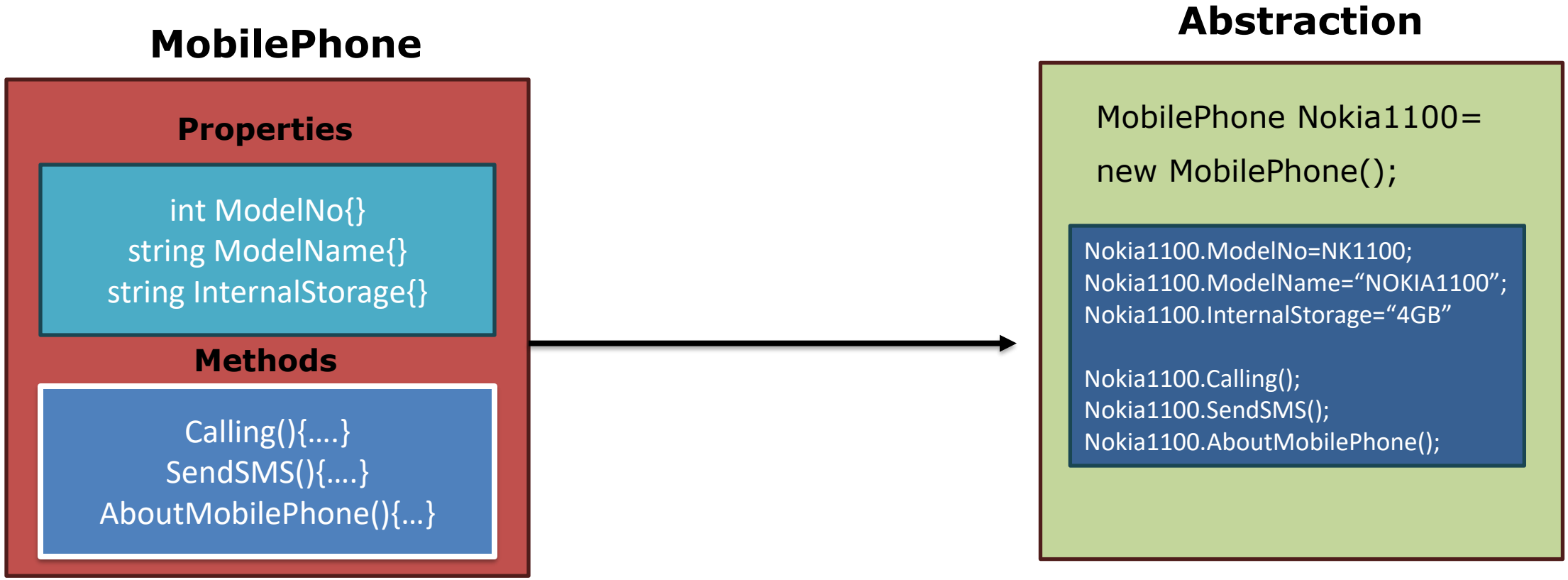
- **Encapsulation: It is the process of hiding irrelevant data from the user**.

- **Inheritance:** is a mechanism that allows one class to inherit properties, methods, and behavior from another class.

- **Polymorphism:** It is the ability of an object to take on different forms or exhibit multiple behaviors based on its context.

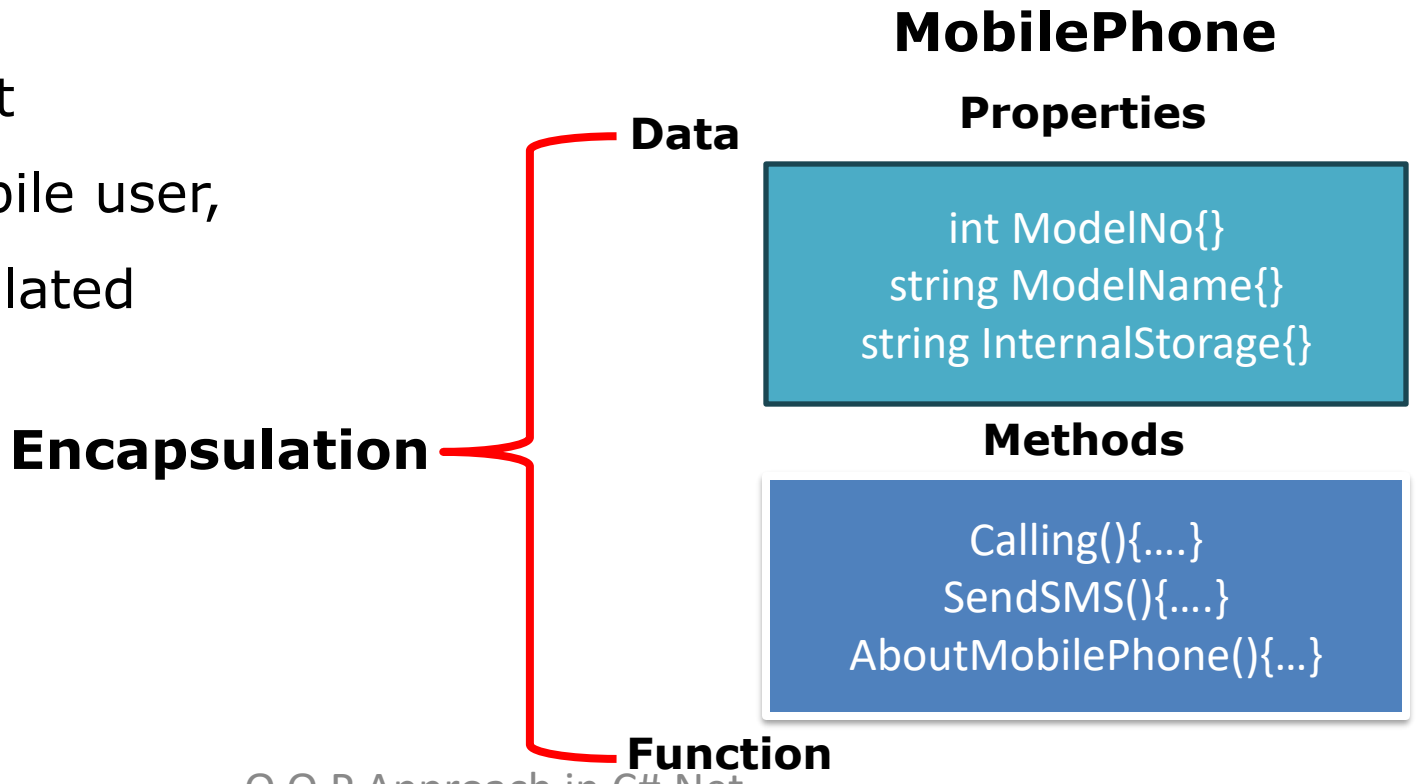
What is Abstraction?

- Abstraction is mechanism to show only relevant data to the user.
- It is the process of hiding the internal details of an application from the outer world.
- It is abstraction, because you are seeing only relevant information instead of their internal engineering.



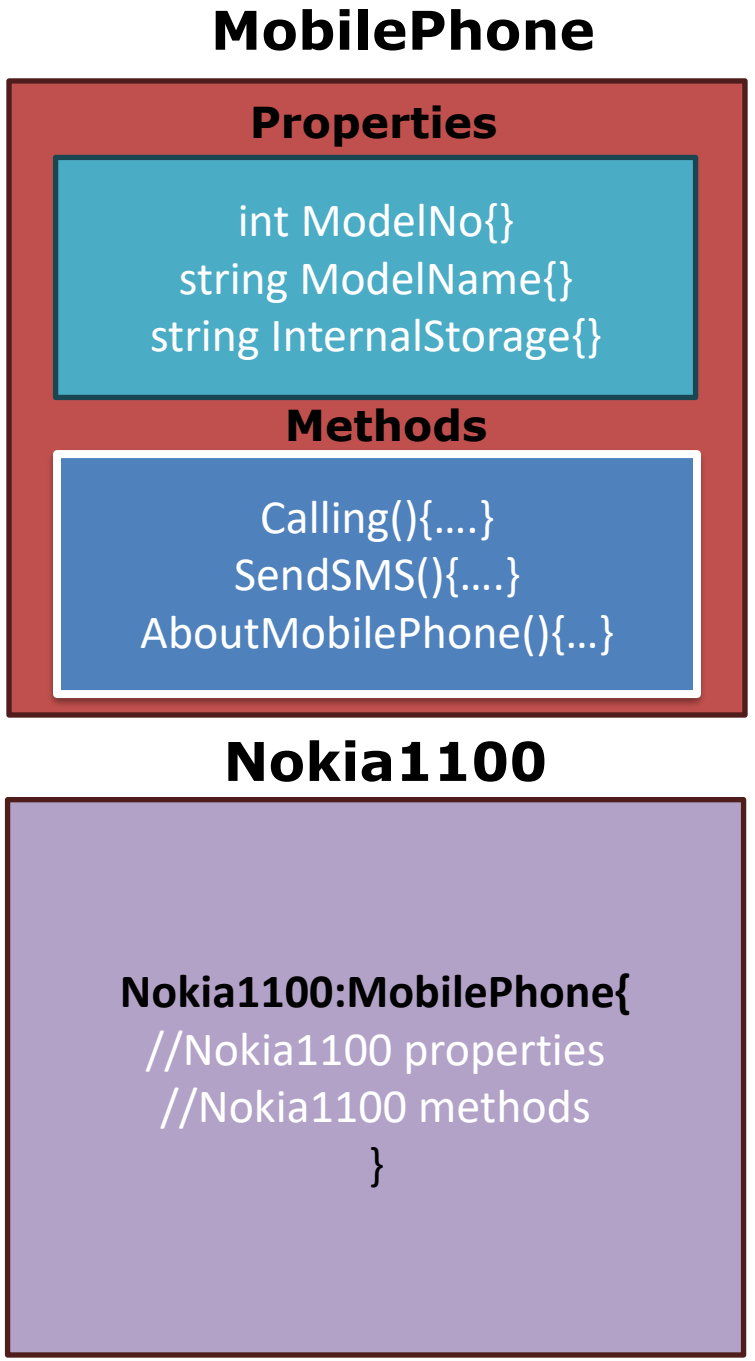
What is Encapsulation?

- Encapsulation is the process of hiding irrelevant data from the user.
- To understand encapsulation, consider an example of mobile phone.
- Whenever you buy a mobile, you don’t see how circuit board works.
- You are also not interested to know how digital signal converts into analog signal and vice versa.
- These are the irrelevant information for the mobile user, that’s why it is encapsulated inside a cabinet.



What is Inheritance?

- Inheritance in object-oriented programming (OOP) is a mechanism that allows a class (known as the derived class or subclass) to acquire properties, methods, and behaviors from another class (known as the base class or superclass).
- It is a way of creating a hierarchy or relationship between classes, where the derived class inherits the characteristics of the base class and can extend or modify them as needed.
- Inheritance promotes code reuse and modularity by allowing the derived class to inherit the members (fields, properties, methods) of the base class.

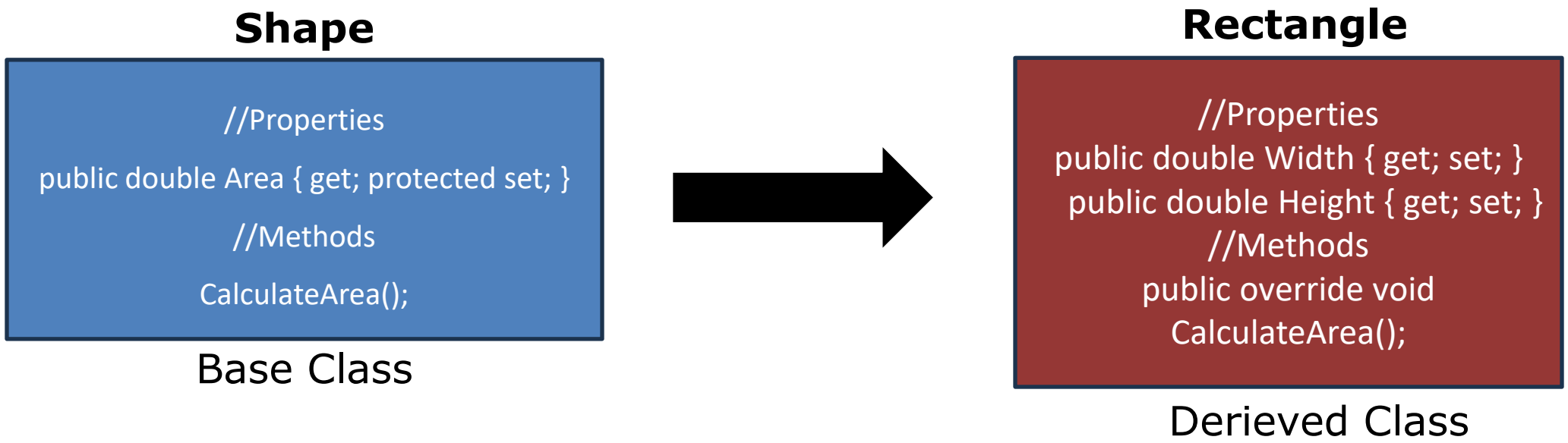


Types of Inheritance:

- **Single Inheritance:**

Single inheritance is a concept in object-oriented programming (OOP) where a derived class or subclass inherits properties, methods, and behaviors from a single base class or superclass.

In single inheritance, a class can have only one direct parent class.



```
using System;

public class Shape
{
    public double Area { get; protected set; }

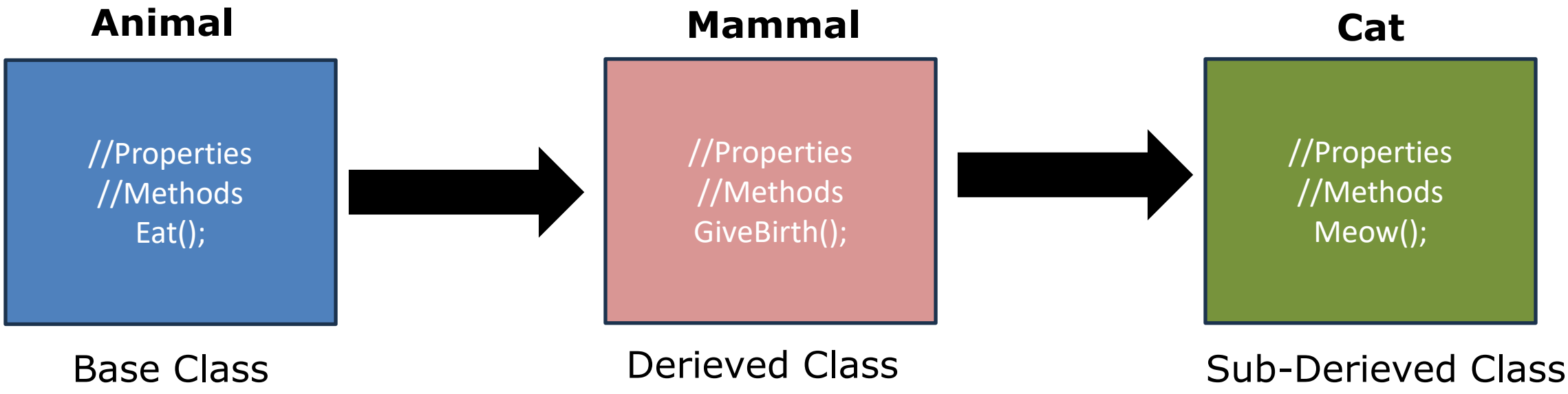
    public virtual void CalculateArea()
    {
        Console.WriteLine("Calculating area of the
shape...");
    }
}

public class Rectangle : Shape
{
    public double Width { get; set; }
    public double Height { get; set; }

    public override void CalculateArea()
    {
        Area = Width * Height;
        Console.WriteLine("Calculating area of the
rectangle...");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Rectangle rectangle = new Rectangle();
        rectangle.Width = 5;
        rectangle.Height = 3;
        rectangle.CalculateArea();
        Console.WriteLine("Area of the rectangle: " +
rectangle.Area);
    }
}
```

- **Multi Level:**Multi-level inheritance is a concept in object-oriented programming (OOP) where a derived class or subclass inherits properties, methods, and behaviors from a base class or superclass, and in turn, serves as the base class for another derived class.



```
using System;

public class Animal
{
    public void Eat()
    {
        Console.WriteLine("Animal is eating.");
    }
}

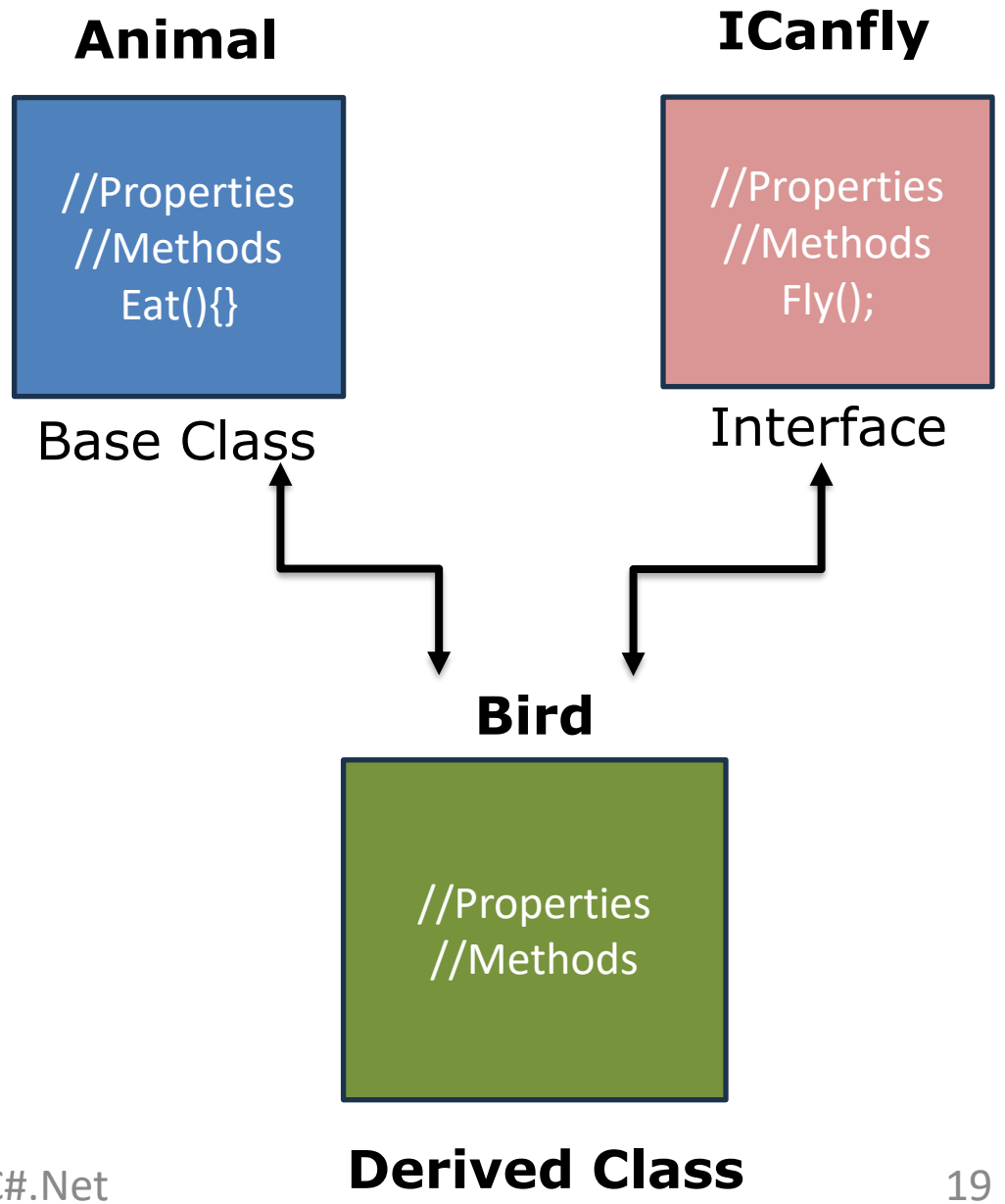
public class Mammal : Animal
{
    public void GiveBirth()
    {
        Console.WriteLine("Mammal is giving birth.");
    }
}

public class Cat : Mammal
{
    public void Meow()
    {
        Console.WriteLine("Cat says meow.");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Cat cat = new Cat();
        cat.Eat();       // Inherited from Animal
        cat.GiveBirth(); // Inherited from Mammal
        cat.Meow();      // Specific to Cat
    }
}
```

- **Multiple:**Multiple inheritance refers to a feature in some programming languages where a class can inherit properties and methods from multiple base classes.

  However, multiple inheritance is not supported in C#. C# only supports single inheritance, meaning a class can inherit from a single base class.



```
using System;

public interface IAnimal
{
    void Eat();
}

public interface ICanFly
{
    void Fly();
}

public class Bird : IAnimal, ICanFly
{
    public void Eat()
    {
        Console.WriteLine("Bird is eating.");
    }

    public void Fly()
    {
        Console.WriteLine("Bird is flying.");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Bird bird = new Bird();
        bird.Eat();
        bird.Fly();
    }
}
```

Why Multiple base classes are not supported in C#?

- **Ambiguity and Diamond Problem:** Multiple inheritance can lead to ambiguity and conflicts when two or more base classes define members with the same name. This is known as the "diamond problem" and can create complexities in resolving which inherited member should be used. C# avoids this problem by enforcing single inheritance, which ensures clear and unambiguous member access.
- **Code Clarity and Maintainability:** Multiple inheritance can make code more difficult to understand and maintain, especially when dealing with complex inheritance hierarchies. It can introduce complexities in determining the origin of a particular behavior or property, making code more error-prone and challenging to debug.

- **Interface-Based Approach:** C# promotes the use of interfaces to achieve similar functionality as multiple inheritance. Interfaces allow classes to implement multiple contracts, enabling them to exhibit behaviors from multiple sources without introducing the complexities associated with multiple inheritance. This approach provides a more flexible and manageable way to achieve code reuse and polymorphism.

- **Simplicity and Language Design:** C# was designed to be a simple, general-purpose language that focuses on readability, maintainability, and ease of use. By omitting multiple inheritance, the language aims to keep the syntax and semantics straightforward, making it easier for developers to understand and work with C# code.

- **Hybrid:**Hybrid inheritance is a combination of different types of inheritance, such as single inheritance, multiple inheritance, and multilevel inheritance.
  - It is a concept where a class inherits properties and methods from multiple base classes, forming a complex inheritance structure.
  - Inheritance types, such as single inheritance, multiple inheritance, and multilevel inheritance, are well-defined and widely recognized in object-oriented programming.
  - However, the term "hybrid inheritance" does not have a standardized meaning or specific definition.

What is Polymorphism?

Polymorphism means **one name many forms**. One function behaves different forms.

In other words, "Many forms of a single object is called Polymorphism.“

A Person working as en employee in the company but at home same person act as family member.

A Person has 2 forms as an Employee and as a Family member.

C# Example of Polymorphism:

- Function Overloading
- Virtual Function/Function Overriding
- Shadowing

- **Function Overloading :**
    - Multiple function with the same name but with different signatures.
    - The appropriate method is selected based on the number, types, and order of the arguments passed to the method.
    - Return type is not part of signature hence it not affects overloading.

```
using System;

public class Calculator
{
    public int Add(int num1, int num2)
    {
        return num1 + num2;
    }

    public double Add(double num1, double num2)
    {
        return num1 + num2;
    }

    public string Add(string str1, string str2)
    {
        return str1 + str2;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Calculator calculator = new Calculator();

        int sum1 = calculator.Add(2, 3);
        Console.WriteLine("Sum1: " + sum1);                // Output: Sum1: 5

        double sum2 = calculator.Add(2.5, 3.5);
        Console.WriteLine("Sum2: " + sum2);                // Output: Sum2: 6.0

        string concatenatedStr = calculator.Add("Hello", " World");
        Console.WriteLine("Concatenated String: " + concatenatedStr); // Output: Concatenated String: Hello World
    }
}
```

- **Virtual Function/Function Overriding :**

  - If a function or a property in the base class is declared as virtual it can be overridden in any derived classes.
  - Virtual function is use to change functionality of base class function in its
  - derived class.
  - By using function overriding, we can define common methods in the base class and provide specialized implementations in the derived classes.
  - This allows for polymorphic behavior, where objects of different derived types can be treated as objects of the base class but exhibit different behaviors based on their actual types.

Keywords: virtual [Base Version]
override [Derived Version]

```
using System;

public class Shape
{
    public virtual double CalculateArea()
    {
        return 0;
    }
}
public class Circle : Shape
{
    private double radius;

    public Circle(double radius)
    {
        this.radius = radius;
    }
    public override double CalculateArea()
    {
        return Math.PI * radius * radius;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Shape shape1 = new Circle(5);
        double circleArea = shape1.CalculateArea();
        Console.WriteLine("Circle Area: " + circleArea);  //
Output: Circle Area: 78.53981633974483


    }
}
```

- **Shadowing:**

  - Function shadowing is a concept where a derived class declares a method with the same name as a method in its base class, effectively hiding the base class method within the derived class.
  - This concept is used for suppress version of base class using **new** keyword.
  - new keyword is not mandatory it is only to denote new version of a function.

```
using System;

public class Shape
{
    public void Draw(string color)
    {
        Console.WriteLine($"Drawing a shape with
{color} color");
    }
}

public class Circle : Shape
{
    public new void Draw(string color, double
radius)
    {
        Console.WriteLine($"Drawing a circle with
{color} color and radius {radius}");
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Shape shape1 = new Shape();
        shape1.Draw("red");  // Output: Drawing a shape
with red color

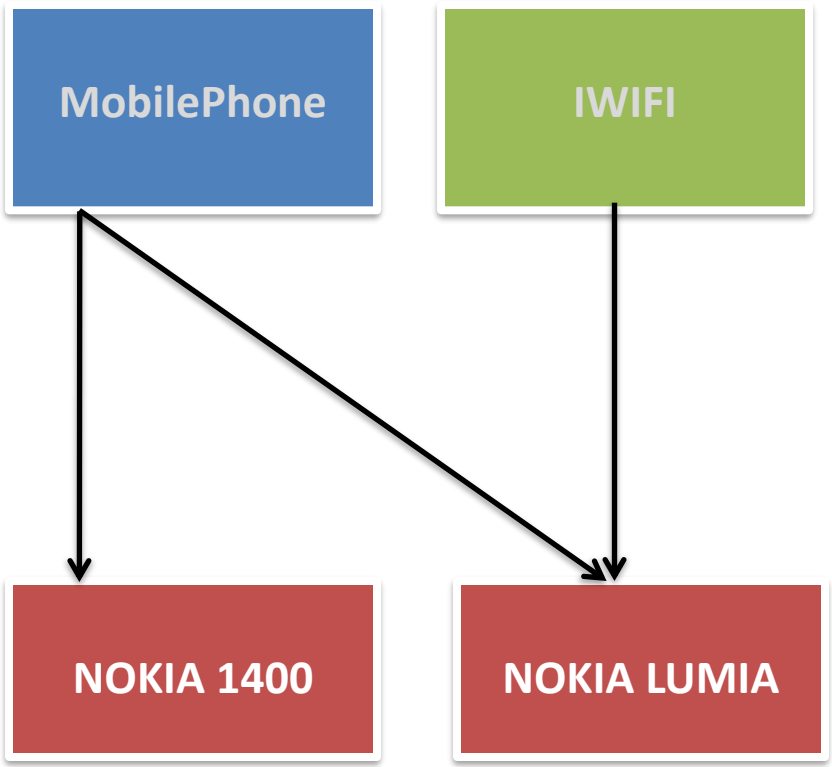
        Circle circle = new Circle();
        circle.Draw("blue", 5.0);  // Output: Drawing a circle
with blue color and radius 5.0
    }
}
```

Comparison between Function Overloading, Overriding and Shadowing.

Term	Overloading	Overriding	Shadowing
Scope	both versions must Be in the same class.	Version must be separated in base class and derived class.	Version must be separated in base class and derived class.
Proto	Same name but with different signatures.	Name and signature must be same of both versions.	Name should be same but signatures can be different or signatures can also be same.
Keyword	No Keyword Required.	Keywords :-Virtual and override.	No Keyword Required.
Required	Not mandatory to overload all function.	Not mandatory.	No Shadowing mandatory for all functions.
Return Type	Return Type not matters.	Return Type matters.	Return Type not matters.

What is Interfaces?

- An Interface is simply a collection of function and prop. prototypes.
- An Interface defines a contract. A class that implements an interface must adhere to that contract.
- An Interface Can inherit from multiple base interfaces and a class.
- Interfaces are reference types.
- An interface is like an abstract class, except it does not contain any logic.
- it doesn’t have any constructor and its has only declarations of functions.
- by default all members are public inside interface hence no need declare function with public keyword explicitly.
- interfaces can only contains declaration of functions and prop.
- it doesn't contain any variable or constant.
- All functionalities needs to be implemented in derived class which is declared Inside interface.
- There are two kind of implementation-implicit and explicit.


MobilePhone
IWIFI
NOKIA 1400
NOKIA LUMIA

```
using System;

namespace OOPDemo
{
    internal interface IWifi
    {
         string StartWifi();
        string StopWifi();

        string TroubleShoot();
    }
}
```

```
using System;

namespace OOPDemo
{
    //Multiple inheritance
    internal class NokiaLumia : MobilePhone, IWifi
    {
        //Explicit implementation
        //string IWifi.StartWifi()
        //{
        //    return "Startting Wifi Functionality";
        //}

        //string IWifi.StopWifi()
        //{
        //    return "Stopping Wifi functionality";
        //}
        //string IWifi.TroubleShoot()
        //{
        //    throw new NotImplementedException();
        //}
```

```
public string PushMessage()
        {
            return "This is PushMessage from
NokiaLumia ";
        }
        //Implicit Implementation
        public string StartWifi()
        {
            return "Startting Wifi Functionality";
        }
        public string StopWifi()
        {
            return "Stopping Wifi functionality";
        }
        public string TroubleShoot()
        {
            throw new NotImplementedException();
        }
    }
}
```

```
using System;
namespace OOPDemo
{
    //This program demo how to use interface?
    internal class InterfaceDemo
    {
        static void Main()
        {
            //calling explicitly implemented
functionalities
            //IWifi wifi = new NokiaLumia();

            //Console.WriteLine(wifi.StartWifi());
            //Console.WriteLine(wifi.StopWifi());
            //Console.WriteLine(wifi.TroubleShoot());//throws
an exception

            //calling implicitly implemented
functionalities

            NokiaLumia nokiaLumia = new NokiaLumia();

            Console.WriteLine(nokiaLumia.Calling());
            Console.WriteLine(nokiaLumia.StartWifi());
            Console.WriteLine(nokiaLumia.StopWifi());
            Console.WriteLine(nokiaLumia.PushMessage());
        }}}
```

Comparison between abstract class and Interface.

Term	Abstract class	Interface
Content	It can contain declaration and implementation too.	It can not contain implementation. It only contains declaration.
Default modifier	All member of abstract class are private by default.	All member of Interface are by default public.
Restriction	It can contain const,field members, prop. and functions.	Only contain prop. and methods .

When to use Interface instead of abstract class?

- When you want to define a contract or a set of capabilities that a class must implement, without specifying any default behavior.

- When you need to enable multiple inheritance of interface types, as C# does not support multiple inheritance of classes.

- When you want to establish a loosely coupled design by programming to interfaces rather than concrete classes, enabling easier unit testing, dependency injection, and code maintainability.

What is Constructor?

When you use the *new* operator it may be necessary to initialize field values to custom values. It is not necessary to define a C# constructor a default constructor is called if there are no programmer defined constructors. The default constructor does not do anything except creates your object.

- Constructor can be Inherited.
- Constructor can be Overload.
- Constructor does not have any return type.
- Constructor must be public in order create an object of a class.

Destructors in C# are methods inside the class used to destroy instances of that class when they are no longer needed. The Destructor is called implicitly by the .NET Framework's Garbage collector and therefore programmer has no control as when to invoke the destructor. An instance variable or an object is eligible for destruction when it is no longer reachable.

- A Destructor is unique to its class i.e. there cannot be more than one destructor in a class.
- A Destructor has no return type and has exactly the same name as the class name (Including the same case).
- It is distinguished apart from a constructor because of the Tilde symbol (~) prefixed to its name.
- A Destructor does not accept any parameters and modifiers.
- It cannot be defined in Structures. It is only used with classes.
- It cannot be overloaded or inherited.
- It is called when the program exits.
- Internally, Destructor called the Finalize method on the base class of object.

```
using System;
namespace OOPDemo
{
    internal class Student
    {
        public int RollNo { get; set; }

        public string Name { get; set; }

        public Student(int RollNo)
        {
            Console.WriteLine($"Student
             Overloaded Constrcutor:{RollNo}");
        }

        public Student(int RollNo,string Name)
        {
            this.RollNo = RollNo;
            this.Name = Name;
        }
```

```
 public Student(Student student)
        {
            this.RollNo = student.RollNo;
            this.Name = student.Name;
        }

        //destructor function-invokes immediately after
closing the application
        ~Student()
        {

        }

    }
}
```

What is Property?

A program needs a way to access and manipulate **private** variables in a class.

- To do this we would rely on **get** and **set** accessors in C# **properties**.
- C# **properties** are blocks that give the client the ability to *set* and *get* private variables in a class.
- Properties can be ReadOnly or ReadWrite.
- Properties can be Parameterized or Parameterless.
- get/set does not have any access modifier they are public by default.
- Properties should be public.
- Property can be of any type.

```
public class <Class Name>
  {
     private <type> <variable>;

     public  <return type> <property name>[ param]
         {
           get
              {
                return <private variable>;
               }
            set
                {
                <private variable> = value;
                }
          }
  }
```

```
using System;
namespace OOPDemo
{
    internal class Product
    {
        //Auto Implemented Properties

        public int ProCode { get; set; }

        public string ProName { get; set; }

        public DateTime ExpiryDate { get; set; }

        //Manually Implemented properties
        private string _ProCategory;

//Read-only Property
        public string MadeIn
        {
            get
            {
                return "India";
            }
        }}
```

```
//Read-Write Property
        public string ProCategory
        {
            get
            {
                return _ProCategory;
            }
            set
            {
                if (value != null)
                {
                    _ProCategory = value;
                }
                else
                {
                    _ProCategory = "Unknown";
                }
            }
        }
```

```
 static void Main()
        {
            Product product1 = new Product { ProCode = 100, ProName = "Tooth Paste",ExpiryDate=
DateTime.Parse("04-03-2023"), ProCategory = null };
            Console.WriteLine($"{product1.ProCode}\t{product1.ProName}\t{product1.ProCategory}");

}
```

Class Access Modifiers

Scope Modifiers

Sr.No	Modifiers	Scope
1.	Public	Entire assembly/project and out side project as well.
2.	internal (by default)	Within assembly/project only.

Functional Modifiers

Sr. No	Modifiers	Description
1.	abstract	Must inherit. can not instantiate.
2.	sealed	Can not inherit must be instantiate.
3.	static	Not specified with any instance. Can not inherit.
4.	partial	It can split class implementation into multiple files using the partial keyword. Each partial class file contains a part of the class definition, and all the parts together form a single logical class.

Abstract Class

What is an Abstract class?

In C#, an abstract class is a class that cannot be instantiated directly and is intended to serve as a base or blueprint for other classes. It provides a way to define common functionality and structure that can be inherited and extended by derived classes.

- **Cannot be instantiated:** An abstract class cannot be directly instantiated using the new keyword. It is designed to serve as a base class or blueprint for other classes.
- **Provides a common interface:** An abstract class typically defines a common interface or contract that derived classes must follow. It often includes abstract methods that derived classes must implement.
- May contain **abstract** and **concrete** members.
- **Abstract methods must be overridden.**

When to use Abstract classes instead of Interface?

The decision to use an abstract class or an interface depends on the specific requirements and design goals of your application. Here are some scenarios where you might prefer using an abstract class over an interface:

- **When you want to provide a default implementation:** An abstract class can include both abstract and non-abstract (concrete) methods, allowing you to provide a default implementation for certain methods. Concrete classes that inherit from the abstract class can choose to override these methods or use the default implementation. This can be useful when you have common behavior that you want to share among multiple derived classes.

- **When you need to define shared state or member variables:** An abstract class can have member variables, constructors, and other class elements, allowing you to define shared state among derived classes.

- **When you want to evolve the class hierarchy over time:** An abstract class provides a structured way to extend and evolve the class hierarchy. You can add new methods or members to the abstract class in future versions without breaking existing derived classes. In contrast, if you add a new method to an interface, all classes implementing that interface would need to provide an implementation for the new method, which can be cumbersome and may require modifying a lot of code.
- **When you need to achieve a mix of reusability and consistency:** Abstract classes allow you to define a common base implementation while also providing the flexibility to override specific methods in derived classes. This can be helpful when you want to enforce a consistent behavior across a group of related classes while allowing some degree of customization.

```
class Circle : Shape
{
    private int radius;

    public Circle(int x, int y, int radius) : base(x, y)
    {
        this.radius = radius;
    }

    public override void Draw()
    {
        Console.WriteLine("Drawing a circle at ({0}, {1}) with radius {2}", x, y, radius);
    }
}
```

```
class Circle : Shape
{
    private int radius;

    public Circle(int x, int y, int radius) : base(x, y)
    {
        this.radius = radius;
    }

    public override void Draw()
    {
        Console.WriteLine("Drawing a circle at ({0}, {1}) with radius {2}", x, y, radius);
    }
}
```

```
Shape shape = new Circle(10, 20, 5);
shape.Draw(); // Calls the Draw() method of the Circle class
```

Sealed Class

What is Sealed class?

In C#, a sealed class is **a class that cannot be inherited** or used as a base class for other classes. It is the opposite of an abstract class, which is designed to be inherited and extended. By marking a class as sealed, you prevent other classes from deriving from it.

```
class Discount
   {
      public double ProAmount { get; set; }

      public double Per { get; set; }

      public Discount(double ProAmount,double Per)
      {
         this.ProAmount = ProAmount;
         this.Per = Per;
      }
      public double GetDiscount()
      {
         double discAmount = ProAmount / 100 * Per;
         return discAmount;
      }
      public virtual double GetProNetAmount()
      {
         double netAmount = ProAmount - GetDiscount();
         return netAmount;
      }
   }
```

```
sealed class FlatDiscount : Discount
{
    public FlatDiscount(double proAmount,double per):base(proAmount,per)
    {

    }

    public sealed override double GetProNetAmount()
    {
        Per = 10;
        double discAmount = ProAmount / 100 * Per;
        double netAmount = ProAmount - discAmount;
        return netAmount;
    }
}
```

```
static void Main()
    {
        FlatDiscount flatDiscount = new FlatDiscount(10000, 10);
        var netAmount = flatDiscount.GetProNetAmount();
        Console.WriteLine(netAmount);
    }
```

When to use Sealed classes?

- **Preventing unintended inheritance:** By sealing a class, you explicitly state that it is not intended to be a base class. This can help avoid potential issues that may arise from unintended inheritance, such as unexpected behavior or violation of the class's design principles.
- **Ensuring implementation integrity:** Sealing a class allows you to have control over the implementation and behavior of the class. By preventing other classes from inheriting from it, you can ensure that the class's functionality remains intact and is not overridden or modified in a way that could compromise its intended behavior.
- **Performance optimizations:** Sealing a class can sometimes provide performance benefits. The runtime can make certain optimizations knowing that the class cannot be extended or overridden, which can result in faster method invocations and improved efficiency.

Static Class

What is a Static class?

In C#, a static class is a class that cannot be instantiated, and all of its members and methods are static. The static keyword is used to declare a class as static. A static class is commonly used to hold utility methods, constants, or shared resources that can be accessed without creating an instance of the class.

- **They cannot be instantiated:** Since a static class cannot be instantiated, you cannot create an instance of it using the new keyword. All access to its members is done through the class name itself.
- **They can only contain static members:** All members (fields, properties, methods) inside a static class must be declared as static. Non-static members, such as instance fields or instance methods, are not allowed in a static class.
- **They are implicitly sealed:** A static class is implicitly sealed, meaning it cannot be inherited or used as a base class. It is a final class that cannot be extended.

```
   static class MathUtils
{
   public static int Add(int a, int b)
   {
      return a + b;
   }

   public static double CalculateCircleArea(double radius)
   {
      return Math.PI * radius * radius;
   }
}
```

```
int sum = MathUtils.Add(2, 3); // Calling the static Add() method

double area = MathUtils.CalculateCircleArea(5.0); // Calling the static CalculateCircleArea() method
```

When to use static classes?

- **Math and utility classes:** Static classes can hold various utility methods, such as mathematical calculations, string manipulation functions, or date/time operations.

- **Helper classes:** Static classes can provide helper methods or functions that can be used across the application without the need to create instances of the class.

- **Constants and configuration:** Static classes can contain constant values or configuration settings that are shared throughout the application.

- **Extension methods:** Static classes can be used to define extension methods that allow adding new functionality to existing types without modifying their original implementation.

Partial Class

What is a Partial class?

In C#, a partial class is a class that can be divided into multiple files, allowing its definition to be spread across different source files within the same namespace. Each part of the partial class contributes to the same final class, and the compiler merges all the parts into a single class during compilation.

- Partial classes are primarily used to organize large classes or classes generated by code generation tools.
- They provide a way to separate the implementation of a class into logical sections, improving code readability and maintainability.
- **Method, Property, and Field Distribution:** The parts of a partial class can contain methods, properties, fields, events, and nested types. All parts collectively contribute to the final class definition. For example, you can have one part of the class defining fields and properties, while another part contains methods and events.

- **Visibility and Accessibility:** The accessibility modifiers used in partial class parts must be the same. For example, if one part declares a method as public, all other parts must also declare it as public. Access modifiers determine the visibility of members within the class and its derived classes.
- **Limitations:** Partial classes can be inherited, and they cannot be used across different namespaces. Each part must belong to the same namespace and have the same class name. Additionally, a single part cannot define the same member multiple times.

```
public partial class Computer
    {
        //Super
        public Computer()
        {
            Console.WriteLine("This is default
constrcutor of computer class");
        }

        public  string Start()
        {
            return "Startting a Computer";
        }

        public string Stop()
        {
            return "Stopping a Computer";
        }

    }
```

```
 partial class Computer
    {
        //Micro Computers
        public Computer(string name)
        {
            Console.WriteLine($"Computer
Type:{name}");
        }

        public string Reboot()
        {
            return "Rebooting a Computer";
        }
    }
```

When to use Partial classes?

- **Separating auto-generated code:** Partial classes are often used when code generation tools, such as Visual Studio designers or ORM tools, generate code for a class. By using partial classes, you can separate the generated code from custom code and avoid overwriting modifications when the code is regenerated.

- **Team collaboration:** Multiple developers working on the same class can use partial classes to work on different parts simultaneously without interfering with each other's code. Each developer can work on their designated part of the class independently.

- **Readability and organization:** Partial classes allow you to logically group related members, improving the overall organization and readability of the codebase.

References

https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop

https://www.c-sharpcorner.com/UploadFile/mkagrahari/introduction-to-object-oriented-programming-concepts-in-C-Sharp/

https://www.geeksforgeeks.org/basics-of-computer-and-its-operations/

https://www.tutorialsteacher.com/csharp/oop

https://www.tutorialsteacher.com/csharp/oop

https://dotnettutorials.net/lesson/object-oriented-programming-csharp/

https://www.softwaretestinghelp.com/c-sharp/oops-concepts-in-csharp/